

SYSTEM ANALYST

presented by Ayub

Requirements Engineering C

У тебя 50 User Stories. Бюджет на 15. Как выбрать какие делать первыми?

Зачем приоритизировать?

Реальность проектов:

- Время всегда ограничено
- Бюджет всегда ограничен
- Хочется больше чем можем сделать

Приоритизация помогает:

- Делать сначала самое ценное
- Говорить "нет" без обид (есть метод)
- Выпустить продукт быстрее

Без приоритизации:

- Делаем всё подряд → ничего не доделываем
- Релиз откладывается бесконечно

MoSCoW — Метод приоритизации

MoSCoW — разбивка требований на 4 категории:

- **M — Must have**: без этого продукт не работает
- **S — Should have**: важно, но можно отложить
- **C — Could have**: приятно иметь, если время есть
- **W — Won't have**: хорошая идея, но не сейчас

Правило: Must have $\leq$ 60% всех требований

MoSCoW — Пример

Интернет-магазин одежды:

Категория	Требования
Must	Каталог, Корзина, Оформление заказа, Оплата
Should	Избранное, Фильтры, История заказов, Отзывы
Could	Рекомендации, Чат поддержки, Программа лояльности
Won't	AR примерка, AI стилист, Instagram Shopping

Must — без этого нельзя купить

Should — улучшает опыт

Could — приятные фишки

Won't — версия 2.0

Kano Model — Основы

Kano Model — классификация по влиянию на удовлетворённость

3 типа требований:

1. **Must-be (базовые)** — нет → очень плохо, есть → нейтрально
 - Пример: тормоза в машине
2. **Performance (линейные)** — чем больше → тем лучше
 - Пример: скорость доставки
3. **Attractive (восторг)** — нет → нормально, есть → WOW
 - Пример: бесплатный десерт в ресторане

Капо — Примеры

Тип	Требование	Объяснение
Must-be	Положить в корзину и оплатить	Без этого не магазин
Must-be	Сайт работает на телефоне	Базовое ожидание 2026
Performance	Скорость загрузки	Быстрее → лучше
Performance	Количество товаров	Больше выбор → лучше
Attractive	AR примерка одежды	WOW-эффект
Attractive	Доставка за 2 часа	Неожиданно круто!

MoSCoW vs Kano

MoSCoW	Kano	Логика
Must have	Must-be + критичные Performance	Без этого не работает
Should have	Performance	Улучшает продукт
Could have	Attractive	WOW-фичи
Won't have	Любые отложенные	Нет времени/бюджета

Вывод:

- MoSCoW — про КОГДА делать
- Kano — про КАК влияет на пользователя

Module Check

1. Зачем приоритизировать требования?
2. Расшифруй MoSCoW
3. В чём разница Must have и Should have?
4. Назови 3 типа Kano Model
5. Приведи Attractive для банковского приложения

Что лучше для ПЕРВОГО релиза?

- А) 80% машины (кузов + 2 колеса, не едет)
- В) Скейтборд (едет, но не идеал)

MVP — Определение

MVP — Minimum Viable Product (минимально жизнеспособный продукт)

Что это:

- Версия с минимальным набором функций
- Но минимум УЖЕ решает проблему
- Цель: выпустить быстро, получить обратную связь

Ключевые принципы:

- **Minimum** — только необходимое
- **Viable** — работает и даёт ценность
- **Product** — реальный продукт, не макет

MVP — Правильное понимание

НЕПРАВИЛЬНО:

Колесо → Шасси → Машина без крыши → Машина

До финала пользователь НЕ МОЖЕТ ЕХАТЬ

ПРАВИЛЬНО:

Скейтборд ✓ → Самокат ✓ → Велосипед ✓ → Мотоцикл ✓ → Машина ✓

На КАЖДОЙ итерации можно ехать!

MVP и MoSCoW

Как определить MVP?

Простое правило: MVP = Must have requirements

Процесс:

1. Собрали все User Stories
2. Приоритизировали по MoSCoW
3. Взяли только Must have
4. Это и есть MVP

Пример:

- **MVP:** Каталог + Корзина + Заказ + Оплата
- **v2.0:** + Избранное + Фильтры + Отзывы
- **v3.0:** + Рекомендации + Чат + Лояльность

MVP — Реальные примеры

Компания	Первый MVP	Сейчас
Airbnb	Фото 3 матрасов	Миллионы объектов
Dropbox	Видео-демо (не продукт!)	Облако для миллиардов
Uber	Только SF, только премиум	Глобальное такси
Instagram	Только фото + фильтры	Stories, Reels, шопинг

Все начинали с простого, но рабочего решения

Module Check

1. Расшифруй MVP
2. Почему "колесо → шасси → машина" это НЕ MVP?
3. Как связаны MVP и MoSCoW?
4. Придумай MVP для "Заказ еды"
5. Зачем выпускать MVP, а не ждать идеала?

День 1: "Сделайте каталог из 100 товаров"

День 10: "А добавим фильтры!"

День 20: "А личный кабинет?"

День 30: "А отзывы с фото?"

День 40: "А Instagram?"

Дедлайн: Не успели!

Что пошло не так?

Score Creep — Определение

Score Creep — неконтролируемое разрастание требований

Признаки:

- Постоянно новые "маленькие" задачи
- "А давайте ещё это, это быстро!"
- Сроки не сдвигаются, работы больше

Почему опасно:

- Сроки срываются
- Бюджет выходит за рамки
- Качество падает
- Команда выгорает

Почему Score Creep происходит?

1. "Это же мелочь!"

- 10 мелочей = большая задача

2. Нет baseline

- Не зафиксировали ЧТО делаем

3. Боимся сказать "нет"

- Соглашаемся на всё

4. Новые идеи в процессе

- "О, а можно ещё так?"

5. Плохая коммуникация

Как предотвратить

1. **Baseline Requirements** — зафиксировать scope
2. **Change Request Process** — формальная процедура
3. **"Да, но"** — "Можем, но +2 недели"
4. **Trade-offs** — "Фильтры или личный кабинет?"
5. **Регулярные встречи** — напоминать scope

Change Request Process

Change Request (CR) — формальный процесс изменений

Шаги:

1. **Заявка** — ЧТО хочет и ЗАЧЕМ
2. **Оценка** — время, деньги, риски
3. **Решение** — добавить или отложить
4. **Документирование** — обновить baseline

Пример:

ID: CR-001

Что: Фильтр по размеру

Зачем: 60% ищут по размеру (аналитика)

Оценка: 3 дня разработки + 1 тест

Решение: Одобрено, +4 дня к дедлайну

Module Check

1. Что такое Scope Creep?
2. Почему опасен?
3. Назови 3 причины Scope Creep
4. Что такое Baseline Requirements?
5. Зачем Change Request Process?

Представь: Ты заказываешь пиццу. Опиши ВСЕ шаги от "проголодался" до "поел".

Где разочаруешься? Где обрадуешься?

CJM — Customer Journey Map

CJM — карта пути клиента через продукт

Что показывает:

- Все этапы взаимодействия
- Действия на каждом этапе
- Touchpoints — где "касается" бренда
- Эмоции (позитив/негатив)
- Pain Points — проблемы

Зачем SA:

- Найти где улучшить
- Видеть полную картину
- Приоритизировать боли

Компоненты CJM

1. Этапы (Stages)

- Осознание → Рассмотрение → Покупка → Использование

2. Действия (Actions)

- Что пользователь делает

3. Touchpoints

- Сайт, приложение, email, соцсети

4. Эмоции

- График вверх/вниз

5. Pain Points

Где спотыкается

CJM — Пример "Заказ еды"

Этап	Pain Point	Opportunity
Выбор	Долго грузится	Загрузка < 2 сек
Заказ	Ввод адреса каждый раз	Сохранённые адреса
Ожидание	Не вижу курьера	Трекинг на карте
Получение	Еда остыла	Термосумка

От CJM к требованиям

Шаг 1: Найди Pain Points

- "Долго загружается"
- "Ввожу адрес каждый раз"

Шаг 2: Opportunities

- Загрузка < 2 сек
- Сохранённые адреса

Шаг 3: User Stories

- "Как покупатель, я хочу чтобы каталог грузился < 2 сек"
- "Как клиент, я хочу сохранять адреса"

Шаг 4: Приоритизируй (MoSCoW)

Module Check

1. Что такое CJM и из чего состоит?
2. Что такое Pain Points?
3. Приведи 3 примера Touchpoints
4. Как превратить Pain Point в User Story?
5. Зачем SA строить CJM?

Requirements Traceability

Traceability — отслеживание связей

Связи:

```
Business Req → User Req → FR → Test Cases
```

Зачем:

- Ничего не потеряется
- Понятно ЗАЧЕМ каждое требование
- Легко найти что не реализовано
- Оценить влияние изменений

Traceability Matrix

Business	User	FR	Test
BR-001: Продажи +25%	UR-001: Найти товар	FR-001: Фильтр цены	ТС-001: Фильтр работает
BR-001: Продажи +25%	UR-002: Добавить в корзину	FR-002: Кнопка корзины	ТС-002: Товар в корзине
BR-002: Меньше звонков	UR-003: Отследить заказ	FR-003: Статус заказа	ТС-003: Статус обновляется

Читаем: Бизнес хочет продажи → пользователь должен искать → нужен фильтр → проверяем тестом

Stakeholder Register

Stakeholder — заинтересованная сторона

Что записываем:

- Имя, роль
- Интерес в проекте
- Влияние (высокое/среднее/низкое)
- Как вовлекать

Имя	Роль	Интерес	Влияние
Иванова	Владелец	Продажи, ROI	Высокое
Покупатель	Пользователь	Удобство, доставка	Среднее
Команда	Разработчики	Чёткие требования	Среднее

Final Check: Урок С

1. Объясни MoSCoW + по 1 примеру
2. Must-be vs Attractive (Kano)?
3. Нарисуй MVP подход (скейт → ...) — почему правильно?
4. Scope Creep и Change Request Process?
5. Зачем CJM? Как помогает собрать требования?
6. Requirements Traceability — что с чем связываем?

Ресар: Урок С

Концепция	Суть
MoSCoW	Must/Should/Could/Won't
Kano	Must-be, Performance, Attractive
MVP	Минимальный рабочий продукт
Scope Creep	Бесконтрольный рост scope
Change Request	Формальный процесс изменений
CJM	Карта пути клиента
Traceability	Business → User → FR → Test
Stakeholder Register	Таблица заинтересованных сторон

| *SA не просто собирает требования — он управляет ими.*

Путь требования

1. **Elicitation** — собрали (интервью, CJM)
2. **Документирование** — User Stories + AC
3. **Приоритизация** — MoSCoW, MVP
4. **Управление** — Traceability, CR
5. **Разработка** → **Тестирование** → **Релиз**
6. **Итерация** — добавляем Should, Could

Твоя роль: провести требование от идеи до продакшена

Вопросы?